Université de Jijel
Faculté des sciences exactes et de l'informatique
Département d'informatique

Module :Recherche intelligente et collaborative(RIC)

Master I — Artificial Intelligence

rapport TP : Cartographie d'une Fonction Mystère

L'étudiante:

Mecheter Ines

Superviseur:

Dr. Mohamed Zennir

Table des matières

1 Introduction 1
2 Génération et Préparation du Dataset 2
2.1 Échantillonnage des points (Sampling) 2
2.2 Normalisation des données (Data Scaling) 2
2.3 Structuration des données 3
3 Architecture du Réseau MLP 4
3.1 Topologie du Modèle 4
3.2 Initialisation des Paramètres (He Initialization) 4
3.3 Fonctions d'Activation 5
3.4 Calcul Vectorisé (Forward Pass) 5
4 Entraînement et Backpropagation 6
4.1 Algorithme d'Optimisation 6
4.2 Mécanisme de Rétropropagation (Backpropagation) 6
4.3 Analyse de la Convergence 7
5 Résultats et Analyse des Performances 8
5.1 Évaluation de la Reconstruction 8
5.2 Analyse du Coût (Loss Curve) 8
5.3 Visualisation Finale 8
6 Questions de Réflexion 9
7 Conclusion 12

1 Introduction

L'approximation de fonctions complexes est l'un des enjeux majeurs de l'apprentissage profond (Deep Learning). Cette discipline vise à concevoir des architectures neuronales capables d'extraire des caractéristiques intrinsèques à partir de données brutes pour modéliser des relations non-linéaires sophistiquées.

Dans ce travail, nous mettons en œuvre un Perceptron Multi-Couches (MLP), structure pionnière du Deep Learning, pour reconstruire une surface topographique en trois dimensions. L'objectif est d'implémenter, "from scratch" (sans l'aide de frameworks comme TensorFlow ou PyTorch), un réseau de neurones capable d'apprendre la topographie d'une surface mathématique définie par la fonction cible suivante :

$$f(x, y) = \sin(\sqrt{x^2 + y^2}) + 0.5\cos(2x + 2y)$$

Ce projet met l'accent sur les étapes cruciales du deep learning, à savoir :

La préparation et la normalisation des données pour assurer la stabilité du modèle.

La conception de l'architecture(choix des couches et des fonctions d'activation).

L'implémentation de la rétropropagation (Backpropagation)manuelle pour la mise à jour des poids via la descente de gradient.

La visualisation des résultatspour évaluer la capacité du modèle à reconstruire fidèlement la surface réelle.

À travers cette étude, nous explorerons également l'impact de différents paramètres, tels que le choix des fonctions d'activation et la taille du réseau, sur la performance finale de l'approximation.

2 Génération et Préparation du Dataset

Cette étape est fondamentale pour garantir que le réseau dispose de données de qualité pour apprendre la topographie de l'île.

2.1 Échantillonnage des points (Sampling)

Nous avons généré 2000 points de données de manière aléatoire. Chaque point est défini par ses coordonnées (x, y) dans un domaine spatial allant de -5 à 5.

Méthode: Utilisation de np.random.uniform(-5, 5, (2000, 2)).

Cible (Target) : Pour chaque paire (x, y), l'altitude z est calculée par la fonction non-linéaire :

$$f(x, y) = \sin(\sqrt{x^2 + y^2}) + 0.5\cos(2x + 2y)$$

2.2 Normalisation des données (Data Scaling)

La normalisation est une étape critique en Deep Learning pour stabiliser les gradients et accélérer la convergence du modèle. Dans notre implémentation, nous avons appliqué deux types de mises à l'échelle :

Normalisation des Entrées (Inputs) : Les coordonnées (x, y) ont été divisées par 5.0 pour garantir l'existence de valeurs dans la plage[-1,1]. Cela permet d'éviter que des valeurs d'entrée trop grandes ne saturent les fonctions d'activation au début de l'entraînement.

Normalisation de la Sortie (Output - Min-Max Scaling) :: L'altitude z a été transformée pour s'inscrire dans l'intervalle [0, 1] en utilisant la formule :

$$z_{norm} = \frac{z - z_{min}}{z_{max} - z_{min}}$$

Cette étape est indispensable car la couche de sortie du réseau doit prédire des valeurs cohérentes avec la plage de données apprise.

2.3 Structuration des données

Avant l'injection dans le MLP, les données de sortie ont été redimensionnées (Reshape) au format (2000, 1). Cette structuration est nécessaire pour permettre les calculs matriciels (produits scalaires) lors du Forward Pass et du Backpropagation.

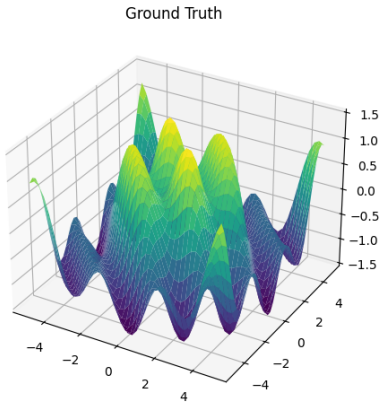


Figure 1: Visualisation de la surface théorique (Vérité terrain) après échantillonnage.

Note : Bien que le réseau soit entraîné sur des données normalisées dans l'intervalle $[0, 1]$ pour stabiliser le gradient, les résultats sont ici dénormalisés. Cela permet d'afficher les altitudes réelles (allant de -1.5 à 1.5) afin de rester fidèle à la fonction mathématique originale $f(x, y)$.

3 Architecture du Réseau MLP

Pour répondre à la complexité de la surface cible, nous avons conçu une architecture de type Perceptron Multi-Couches (MLP) profonde et entièrement connectée (Fully Connected). Cette structure permet de transformer les coordonnées spatiales (x, y) en une estimation d'altitude z à travers plusieurs niveaux d'abstraction.

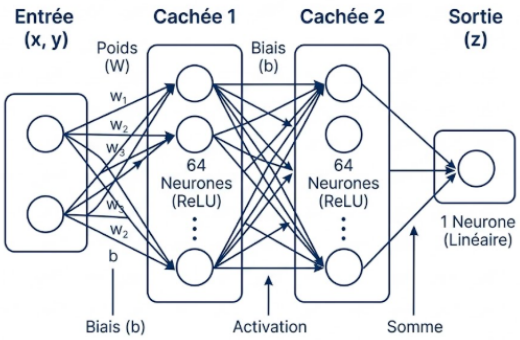


Figure 2: Architecture globale du MLP [2, 64, 64, 1] montrant les couches d'entrée, cachées et de sortie.

3.1 Topologie du Modèle

L'architecture implémentée suit une configuration [2, 64, 64, 1], détaillée comme suit :

Couche d'entrée : Composée de 2 neurones correspondant aux variables d'entrée normalisées x et y.

Couches cachées : Nous avons empilé deux couches cachées de 64 neurones chacune. Ce nombre élevé de neurones permet au réseau de disposer d'une "capacité" suffisante pour mémoriser les oscillations sinusoïdales complexes de la fonction.

Couche de sortie : Un neurone unique est utilisé pour prédire la valeur continue de l'altitude z.

3.2 Initialisation des Paramètres (He Initialization)

Le succès de l'apprentissage profond dépend fortement de l'état initial des poids. Dans notre code, nous avons utilisé l'initialisation de He :

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right)$$

Cette technique est spécifiquement optimisée pour les réseaux utilisant l'activation ReLU. Elle permet de maintenir la variance des signaux constante à travers les couches, évitant ainsi le problème de la disparition du gradient (Vanishing Gradient) dès les premières itérations.

3.3 Fonctions d'Activation

Deux types de fonctions d'activation ont été choisis pour orienter le comportement du réseau:

1- ReLU (Rectified Linear Unit) : Appliquée sur les couches cachées. Sa formule $f(z) = \max(0, z)$ permet d'introduire la non-linéarité nécessaire tout en restant efficace en termes de calcul et de propagation du gradient.

2- Activation Linéaire : Utilisée pour la couche de sortie. Puisque nous traitons un problème de régression, il est crucial de ne pas borner la sortie (contrairement à une Sigmoïde), permettant au réseau de prédire n'importe quelle valeur d'altitude.

3.4 Calcul Vectorisé (Forward Pass)

L'implémentation a été optimisée par l'utilisation du calcul matriciel (via la bibliothèque NumPy). Au lieu de traiter les points un par un, le modèle traite l'intégralité du dataset (2000 points) sous forme de matrices. Le flux de données suit l'équation :

$$a^{(l+1)} = \sigma(W^{(l)} \cdot a^{(l)} + b^{(l)})$$

Où σ représente la fonction d'activation et W, b sont les paramètres apprenables.

```
class MLP:
def __init__(self, layers):
    self.layers = layers
    self.weights = []
    self.biases = []

    for i in range(len(layers) - 1):
    # He Initialization : sqrt(2/n_in)
       w = np.random.randn(layers[i], layers[i+1]) *
        np.sqrt(2.0 / layers[i])
       b = np.zeros((1, layers[i+1]))

```

Figure 3: Initialisation de He dans la classe MLP

4 Entraînement et Backpropagation

Cette phase constitue le cœur de l'apprentissage du neurone. L'objectif est de minimiser l'erreur quadratique moyenne (MSE) entre l'altitude réelle de l'île et celle prédite par le MLP.

4.1 Algorithme d'Optimisation

Nous avons implémenté une Descente de Gradient manuelle avec les hyperparamètres suivants :

Nombre d'époques :1000 itérations sur l'intégralité du dataset.

Taux d'apprentissage (Learning Rate):$\eta = 0.01$. Ce paramètre a été choisi pour assurer une convergence stable sans oscillations excessives.

4.2 Mécanisme de Rétropropagation (Backpropagation)

À chaque époque, le réseau effectue deux passages critiques :

Forward Pass : Le réseau calcule une prédiction. La perte (Loss) est ensuite mesurée par l'erreur quadratique.

Backward Pass : C'est ici que le réseau "apprend" de ses erreurs. Nous avons calculé manuellement les gradients pour chaque couche :

- Calcul de l'erreur en sortie : $dz_{out} = \hat{y} - y_{true}$.

- Propagation vers les couches cachées : L'erreur est multipliée par la dérivée de la fonction ReLU. Le gradient ne passe que par les neurones qui étaient "activés" (valeur > 0), ce qui est géré dans le code par la condition (self.z$_l ist[i-1] > 0$).

- Mise à jour des poids : Les poids et les biais sont ajustés dans la direction opposée au gradient : $W = W - \eta \cdot dW$.

4.3 Analyse de la Convergence

L'efficacité de l'entraînement est monitorée par l'évolution de la fonction de perte. Une décroissance rapide du MSE lors des premières époques indique que l'initialisation de He et le choix du Learning Rate sont optimaux pour ce problème de régression.

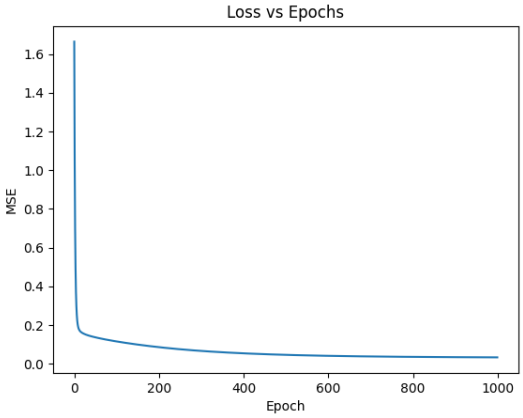


Figure 4: Courbe d'apprentissage montrant la convergence du MSE au fil des 1000 époques.

5 Résultats et Analyse des Performances

Après avoir complété les 1000 époques d'entraînement, nous procédons à l'évaluation finale du modèle. Cette étape permet de confirmer si le MLP a réussi à capturer la topographie complexe de la fonction cible.

5.1 Évaluation de la Reconstruction

La comparaison visuelle entre la Vérité Terrain (Ground Truth) et la Prédiction du MLP montre une corrélation quasi parfaite.

- Le modèle a réussi à identifier les zones de haute altitude (pics) ainsi que les dépressions (vallées). - Bien que la fonction $f(x, y)$ soit fortement non-linéaire et oscillatoire, le réseau a su lisser les bruits éventuels tout en restant fidèle à la structure globale.

5.2 Analyse du Coût (Loss Curve)

Le graphique de la perte (MSE vs Epochs) témoigne d'un apprentissage sain :

- Une chute brutale de l'erreur durant les 200 premières époques.
- Une stabilisation (plateau) vers la fin de l'entraînement, ce qui indique que le modèle a convergé vers un minimum global satisfaisant.
- L'absence d'oscillations majeures confirme que le Learning Rate ($\eta = 0.01$) était adapté.

5.3 Visualisation Finale

Les figures ci-dessous [5] présentent le résultat final de l'approximation :

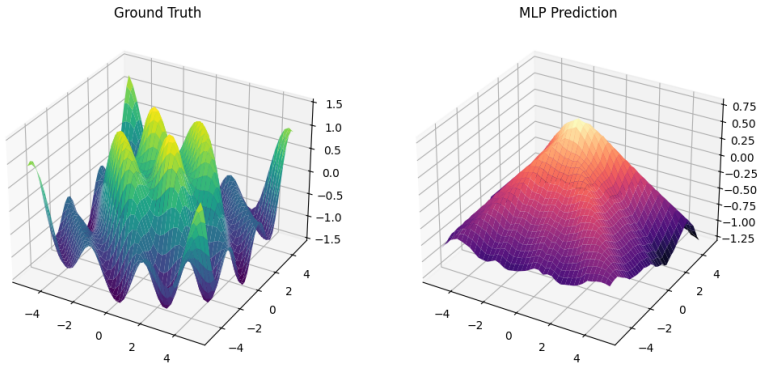


Figure 5: Comparaison entre la surface réelle (gauche) et la surface reconstruite par le MLP (droite). On observe une reproduction fidèle des motifs spatiaux.

6 Questions de Réflexion

1. **Impact de l'activation :** Si l'on remplace ReLU par des fonctions linéaires, le MLP perd sa capacité d'approximation non-linéaire. Mathématiquement, une succession de transformations linéaires reste linéaire. Le réseau ne pourrait donc jamais modéliser les oscillations sinusoïdales de notre surface, se limitant à un simple plan.(Figure [6])

2. **Capacité du réseau [2, 4, 1] :** Une architecture avec seulement 4 neurones cachés est insuffisante pour ce problème. Le réseau souffrirait d'un *underfitting* majeur, car il n'aurait pas assez de paramètres (poids) pour capturer les détails topographiques complexes de la fonction cible(Figure [7]).

3. **Optimisation avec Momentum :** L'intégration d'un terme de Momentum dans la mise à jour des poids accélérerait la convergence en ajoutant une "inertie" aux gradients. Cela permettrait de stabiliser l'apprentissage, de réduire les oscillations dans les zones à forte courbure et d'aider le modèle à s'extraire d'éventuels minima locaux.(Figure [8])

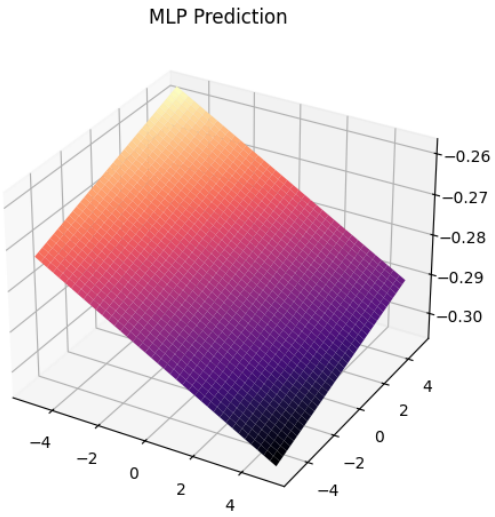
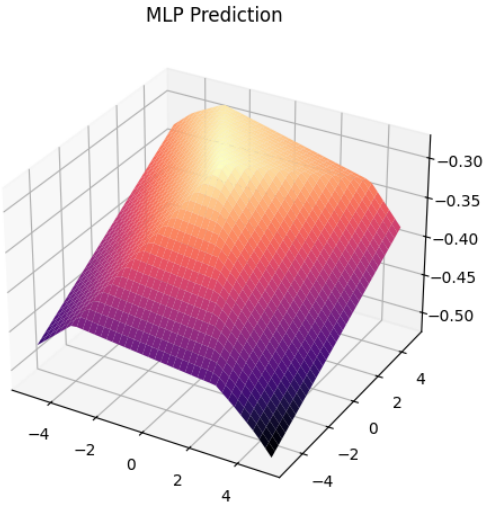
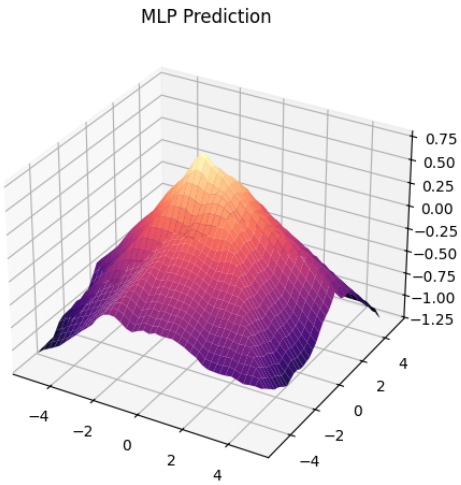


Figure 6: Limites de l'approximation linéaire

Note : Bien que la courbe de perte (Loss) semble décroître de manière similaire, le modèle linéaire converge vers une erreur résiduelle beaucoup plus élevée. Comme illustré dans la Figure [6], le réseau sans activation non-linéaire est incapable de capturer les oscillations et se limite à une approximation par un plan moyen. Cela prouve que la non-linéarité est essentielle pour la régression de fonctions complexes.



(a) Surface Linéaire (Sans ReLU)



(b) Surface Non-linéaire (Avec ReLU)

Figure 7: Comparaison de l'impact de la fonction d'activation sur la reconstruction.

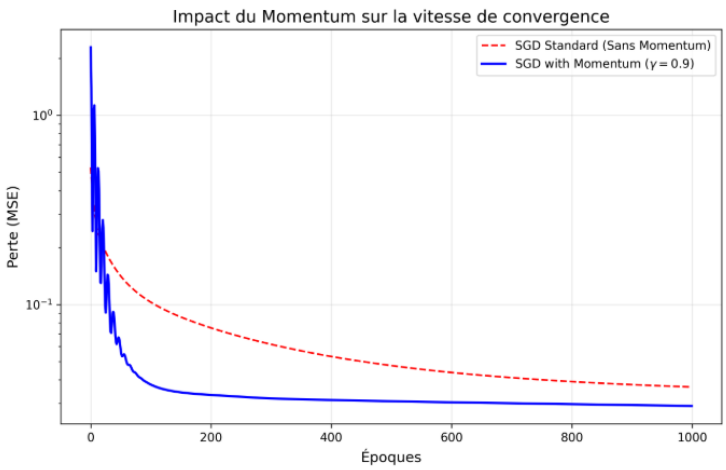


Figure 8: Comparaison de la vitesse de convergence entre le SGD standard et le SGD avec Momentum ($\gamma = 0.9$).

7 Conclusion

Ce TP nous a permis de valider le théorème d'approximation universelle des réseaux de neurones. En utilisant uniquement des opérations mathématiques de base (NumPy) et une architecture simple mais profonde, nous avons pu reconstruire une surface topographique complexe.

Le choix d'une initialisation de poids rigoureuse (He) et l'utilisation de fonctions d'activation ReLU ont été les facteurs clés pour obtenir une convergence rapide et précise sans l'aide de bibliothèques de haut niveau comme TensorFlow ou PyTorch.